



BAHRIA UNIVERSITY, ISLAMABAD
Department of Computer Science

DESIGN AND ANALYSIS OF ALGORITHMS

Assignment # 03

Submitted To: Saima Jawad

Umm-e-Habiba Imran
01-134241-048

1 TABLE OF CONTENTS

1	Problem: 128. Security Switches	3
2	Objective	3
3	Techniques Used.....	4
4	Brute Force.....	4
4.1	Sub Functions	4
4.2	Pseudocode	5
4.3	Complexity Analysis.....	6
4.4	Space Complexity.....	8
5	Divide and Conquer	9
5.1	Recursive Strategy	9
5.2	Pseudocode	10
5.3	Complexity Analysis	10
5.4	Space Complexity.....	12
6	Comparison (Brute Force vs Divide and Conquer).....	13
7	Conclusion	14
8	References	14
9	Acknowledgement of the Implementation	14

1 PROBLEM: 128. SECURITY SWITCHES

There is a row of n security switches protecting a military installation entrance. The switches can be manipulated as follows:

1. The rightmost switch may be turned on or off at will.
2. Any other switch may be turned on or off only if the switch to its immediate right is on and all the other switches to its right, if any, are off.
3. Only one switch may be toggled at a time.

Devise an algorithm to turn off all the switches, which are initially all on, in the minimum number of moves. (Toggling one switch is considered one move.) Find the minimum number of moves.

Initial State: 111...1 (n switches ON)

Goal State: 000...0 (all switches OFF)

- Switch n (rightmost): can always be toggled freely
- Switch i ($i < n$): can be toggled only if switch $i+1$ is ON AND all switches $i+2 \dots n$ are OFF

2 OBJECTIVE

- To design an algorithm that transforms the initial configuration of switches (all ON) into the target configuration (all OFF).
- To ensure that all switch operations strictly follow the given constraints for valid moves.
- To determine the **minimum number of moves** required to turn off all switches.
- To explore all possible valid states in a systematic manner in order to guarantee an optimal solution.
- To avoid revisiting previously explored states by maintaining appropriate data structures.
- To implement and analyze the problem using suitable algorithm design techniques (such as Breadth-First Search and Recursive approach).
- To evaluate the efficiency of the algorithms in terms of time and space complexity.

3 TECHNIQUES USED

1. Brute Force (Breadth-First Search - BFS)

- Explore all valid configurations (states)
- Use BFS to guarantee minimum number of moves

2. Divide-and-Conquer

- Reduce the problem recursively using constraints
- Use recurrence: $M(n) = M(n - 1) + 2M(n - 2) + 1$

4 BRUTE FORCE

The brute force technique is applied by exploring all possible valid states starting from the initial configuration 111...1 to reach the goal state 000...0. Breadth-First Search (BFS) is used to systematically explore the state space level by level, guaranteeing that the first time the goal state is reached corresponds to the minimum number of moves.

4.1 SUB FUNCTIONS

IsValidMove(state, i): Returns true if switch i can be toggled given current state.

- Rightmost switch ($i = n-1$): always valid
- All other switches: valid only if $\text{state}[i+1] = 1$ AND $\text{state}[i+2 \dots n-1]$ are all 0

ToggleSwitch(state, i): Returns a new state with switch i flipped. Does NOT modify original state.

```
Enter number of switches (n): 3
```

```
Building full recursive paths from root = 111 ...  
Total nodes in search tree : 14
```

```
-- Shortest path to target --  
move 0: 111 <- START  
move 1: 110  
move 2: 010  
move 3: 011  
move 4: 001  
move 5: 000 <- TARGET
```

4.2 PSEUDOCODE

Algorithm BFS_SwitchProblem(n)

// Input: n = number of switches

// Output: Minimum number of moves to turn all switches OFF

initial_state $\leftarrow$ [1, 1, 1, ..., 1] // n switches all ON

goal_state $\leftarrow$ [0, 0, 0, ..., 0] // n switches all OFF

if initial_state = goal_state then

 return 0

queue $\leftarrow$ empty FIFO queue

visited $\leftarrow$ {}

Enqueue(queue, (initial_state, 0)) // (state, move_count)

Add initial_state to visited

while queue is not empty do

 (current_state, moves) $\leftarrow$ Dequeue(queue)

 for i $\leftarrow$ 0 to n-1 do // Try toggling each switch

 if IsValidMove(current_state, i) then

 new_state $\leftarrow$ ToggleSwitch(current_state, i)

 if new_state = goal_state then

 return moves + 1 // Minimum moves found

 if new_state $\notin$ visited then

 Add new_state to visited

 Enqueue(queue, (new_state, moves + 1))

return -1 // No solution found (should never happen)

Function IsValidMove(state, i)

// Returns true if switch at position i can be toggled

if i = n-1 then // Rightmost switch

 return true

if state[i+1] $\neq$ 1 then // Switch to right must be ON

 return false

for j $\leftarrow$ i+2 to n-1 do // All switches further right

 if state[j] $\neq$ 0 then

 return false

return true

Function ToggleSwitch(state, i)

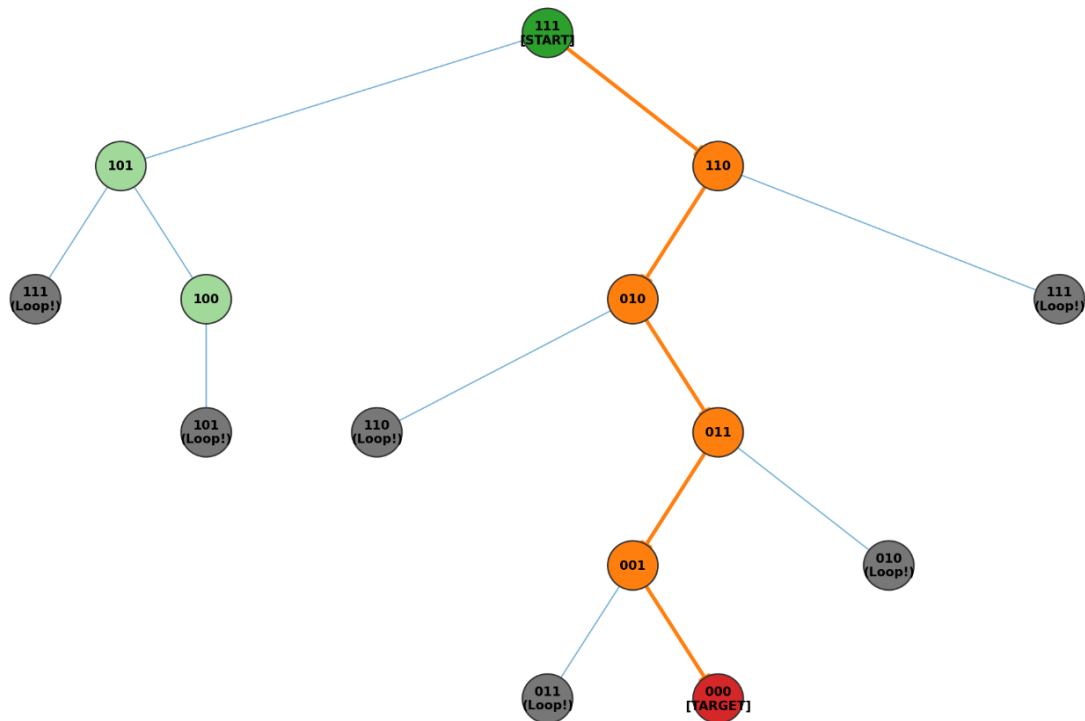
// Returns new state with switch i flipped

new_state $\leftarrow$ copy of state // O(n) copy operation

new_state[i] $\leftarrow$ 1 - new_state[i] // Flip: 0 $\rightarrow$ 1 or 1 $\rightarrow$ 0

return new_state

Full Backtracking Search Tree
Shows why other paths get stuck in a loop!



4.3 COMPLEXITY ANALYSIS

Total Number of States

Each switch has 2 possible values (ON/OFF). For n switches:

$$\text{Total States} = 2^n$$

In the worst case, BFS explores all reachable states before finding the goal.

Cost per State — IsValidMove(state, i)

O(1) per call for boundary checks

Inner loop (checking switches $j = i+2$ to $n-1$):

$$\sum_{k=0}^{n-1} 1 = n - i - 2$$

Cost per State — ToggleSwitch(state, i)

$$\text{Array copy cost} = O(n)$$

Total cost per single state

$$T_{state} = \sum_{i=0}^{n-1} [(n-i-2) + n] = \sum_{i=0}^{n-1} (2n-i-2)$$

Splitting and simplifying the summation:

$$\begin{aligned} &= \Sigma(2n-2) - \Sigma i = n \cdot (2n-2) - n \cdot (n-1)/2 \\ &= 2n^2 - 2n - \frac{n^2 - n}{2} = \frac{4n^2 - 4n - n^2 + n}{2} = \frac{3n^2 - 3n}{2} \\ T_{state} &= (3/2) \cdot n \cdot (n-1) \end{aligned}$$

Total BFS Complexity over all 2^n States

$$T(n) = 2^n \cdot \left(\frac{3}{2}\right) \cdot n \cdot (n-1) = \left(\frac{3}{2}\right) \cdot n(n-1) \cdot 2^n$$

Asymptotic Verification

$$f(n) = \left(\frac{3}{2}\right)(n^2 - n) \cdot 2^n \text{ and } g(n) = n^2 \cdot 2^n$$

$$c_1 \cdot (n^2 \cdot 2^n) \leq \left(\frac{3}{2}\right)(n^2 - n) \cdot 2^n \leq c_2 \cdot (n^2 \cdot 2^n)$$

$$c_1 = \frac{3}{4} \text{ and } c_2 = \frac{3}{2} \text{ for } n \geq 2:$$

$$\left(\frac{3}{4}\right) \cdot n^2 \cdot 2^n \leq \left(\frac{3}{2}\right) \cdot n^2 \cdot 2^n - \left(\frac{3}{2}\right) \cdot n \cdot 2^n \leq \left(\frac{3}{2}\right) \cdot n^2 \cdot 2^n$$

Upper Bound:

$$f(n) = O(n^2 \cdot 2^n)$$

Lower Bound:

$$f(n) = \Omega(n^2 \cdot 2^n)$$

Tight Bound:

$$f(n) = \Theta(n^2 \cdot 2^n)$$

$$T(n) = \Theta(n^2 \cdot 2^n)$$

Best Case vs Worst Case

Worst Case: BFS explores all 2^n states before finding goal $\rightarrow T(n) = \Theta(n^2 \cdot 2^n)$

Best Case: Goal state found immediately (trivial for $n=0$). For non-trivial input ($n \geq 1$) with all switches ON, BFS must traverse $M(n) = \Theta(2^n)$ levels minimum $\rightarrow T(n) = \Omega(2^n)$

Minimum Number of Moves

BFS guarantees optimality — the first path found is always the shortest. From the recurrence:

$$M(n) = M(n-1) + 2 \cdot M(n-2) + 1$$

Even n:

$$M(n) = \frac{(2^{n+1} - 2)}{3}$$

Odd n:

$$M(n) = \frac{2^{n+1} - 1}{3}$$

Minimum Moves:

$$M(n) = \Theta(2^n)$$

4.4 SPACE COMPLEXITY

BFS stores every state it visits in the queue and visited set.

- Total states in the worst case: 2^n
- Each state is stored as an array/string of size n
- Queue and visited set both can grow to 2^n entries simultaneously

$$\text{Space Complexity} = O(n \cdot 2^n)$$

5 DIVIDE AND CONQUER

Instead of exploring all possible configurations, this approach solves the problem **recursively** by reducing it into smaller subproblems, guided by the structural constraints of valid switch operations.

Key Insight: To turn OFF switch 1 (leftmost), the system must first reach the state:

1100...0 (Switch 2 ON, switches 3..n all OFF)

This forces a recursive decomposition into subproblems of size (n-2) and (n-1).

5.1 RECURSIVE STRATEGY

To solve the problem for (n) switches (initially all ON):

Step 1: Turn OFF the last (n-2) switches

1111...1 $\rightarrow$ 1100...0

Cost: **M(n-2)**

Step 2: Toggle switch 1 (now valid since switch 2 is ON and switches 3..n are OFF)

1100...0 $\rightarrow$ 0100...0

Cost: **+1**

Step 3: Restore the last (n-2) switches back to ON

0100...0 $\rightarrow$ 0111...1

Cost: **+M(n-2)**

Step 4: Solve the remaining (n-1) switches recursively

0 | 111...1 $\rightarrow$ 0 | 000...0

Cost: **+M(n-1)**

Final Recurrence Relation

Combining all steps:

$$M(n) = M(n-2) + 1 + M(n-2) + M(n-1)$$

$$M(n) = M(n-1) + 2 \cdot M(n-2) + 1 \quad \text{for } n \geq 3, \quad M(1) = 1, \quad M(2) = 2$$

5.2 PSEUDOCODE

```
Algorithm DivideConquerSwitches(n)
// Input: n = number of switches (all initially ON = 1)
// Output: Minimum number of moves to reach 000...0

if n = 1 then
    toggle switch 1    // Only switch: turn it OFF
    return 1           // Base case: 1 move

if n = 2 then
    toggle switch 2    // Turn OFF rightmost
    toggle switch 1    // Switch 2 OFF, switch 1 can be toggled
    return 2           // Base case: 2 moves

// Step 1: Turn OFF last (n-2) switches [Cost: M(n-2)]
moves ← DivideConquerSwitches(n-2)

// Step 2: Toggle switch 1 [Cost: +1]
toggle switch 1
moves ← moves + 1

// Step 3: Restore last (n-2) switches back to ON [Cost: +M(n-2)]
moves ← moves + DivideConquerSwitches(n-2)

// Step 4: Solve remaining (n-1) switches recursively [Cost: +M(n-1)]
moves ← moves + DivideConquerSwitches(n-1)

return moves
```

5.3 COMPLEXITY ANALYSIS

Recurrence:

$$M(n) = M(n-1) + 2 \cdot M(n-2) + 1$$

Expansion — Step 1: Substitute $M(n-1)$

$$M(n-1) = M(n-2) + 2 \cdot M(n-3) + 1$$

Substitute into M(n):

$$M(n) = [M(n-2) + 2M(n-3) + 1] + 2M(n-2) + 1 = 3M(n-2) + 2M(n-3) + 2$$

Expansion — Step 2: Substitute M(n-2)

$$M(n-2) = M(n-3) + 2M(n-4) + 1$$

Substitute:

$$M(n) = 3[M(n-3) + 2M(n-4) + 1] + 2M(n-3) + 2 = 5M(n-3) + 6M(n-4) + 5$$

General Pattern after k expansions

$$M(n) = a_k M(n-k) + b_k M(n-k-1) + c_k$$

The dominant coefficient leads to:

$$M(n) \approx 2M(n-1) \text{ (dominant term)}$$

Closed-Form Solution (from Book)

$$M(n) = \left(\frac{2}{3}\right) 2^n - \left(\frac{1}{6}\right) (-1)^n - \frac{1}{2}$$

Even n

$$M(n) = \frac{2^{n+1} - 2}{3}$$

Odd n

$$M(n) = \frac{2^{n+1} - 1}{3}$$

Asymptotic Bounds

Upper Bound:

$$M(n) \leq M(n-1) + 2M(n-1) + 1 = 3M(n-1) + 1$$

$$M(n) = O(2^n)$$

Lower Bound:

$$M(n) \geq 2 \cdot M(n-2)$$

$$M(n) = \Omega(2^n)$$

Tight Bound: Since $M(n)$ grows as

$$\left(\frac{2}{3}\right) \cdot 2^n \rightarrow M(n) = \Theta(2^n)$$

$$T(n) = \Theta(2^n) \text{ [Worst Case = Best Case = Only Case]}$$

There is **only one case** for this algorithm — the algorithm always follows exactly the same recursive structure regardless of input, since the initial state is always all-ON and the goal is always all-OFF.

```

=====
      Divide and Conquer - Security Switches Solver
=====
Recurrence: T(n) = T(n-1) + 2*T(n-2) + 1
Base Cases: T(1) = 1, T(2) = 2
-----
Enter number of switches (n): 5

=====
Recurrence Expansion
=====
T(1) = 1
T(2) = 2
T(3) = T(2) + 2*T(1) + 1 = 2 + 2*1 + 1 = 5
T(4) = T(3) + 2*T(2) + 1 = 5 + 2*2 + 1 = 10
T(5) = T(4) + 2*T(3) + 1 = 10 + 2*5 + 1 = 21

=====
Result
=====
Number of switches (n)      : 5
Minimum moves required T(n) : 21
=====

```

5.4 SPACE COMPLEXITY

The algorithm is recursive. Space usage depends on the maximum recursion depth.

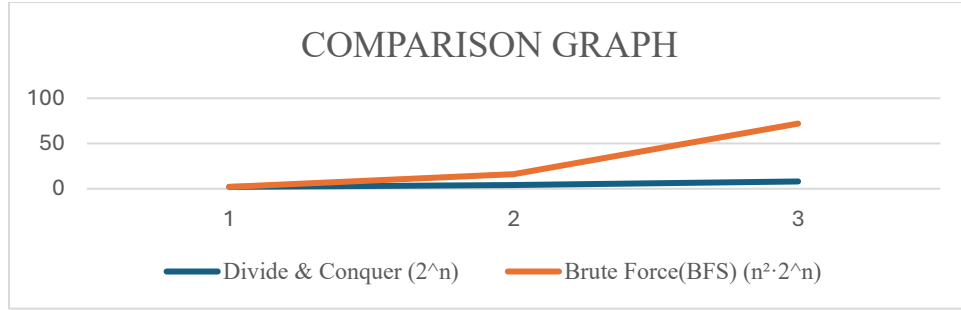
Recursion chain: $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots \rightarrow 1$

Maximum depth: n levels (each call reduces by at least 1)

$$\text{Space Complexity} = O(n)$$

6 COMPARISON (BRUTE FORCE VS DIVIDE AND CONQUER)

Criterion	Algorithm 2: Brute Force (BFS)	Algorithm 1: Divide & Conquer
Design Technique	Brute Force (BFS State-Space Search)	Divide and Conquer (Recursive Decomposition)
Approach	Explore all valid states level-by-level until goal is reached	Exploit constraint structure to directly compute solution path
Worst Case	$\Theta(n^2 \cdot 2^n)$	$\Theta(2^n)$
Best Case	$\Omega(2^n)$ [when goal found early]	$\Theta(2^n)$ [same as worst]
Upper Bound	$O(n^2 \cdot 2^n)$	$O(2^n)$
Lower Bound	$\Omega(n^2 \cdot 2^n)$	$\Omega(2^n)$
Time Tight Bound	$\theta(n^2 \cdot 2^n)$	$\theta(2^n)$
Space Complexity	$O(n \cdot 2^n)$ [queue + visited set]	$O(n)$ [recursion stack depth]
Optimality	Always optimal — BFS guarantees shortest path	Always optimal — directly computes minimum moves
Correctness Guarantee	Guaranteed by BFS level-order exploration property	Proven by induction on recurrence structure
Number of Cases	Best, average, and worst cases differ	Only one case (deterministic structure)
Data Structures	FIFO queue + hash set (visited states)	Call stack (implicit recursion)
Scalability	Less scalable — space is $O(n \cdot 2^n)$, memory-intensive	More scalable — space is only $O(n)$
Knowledge Required	No domain knowledge needed — generic state exploration	Problem insight needed to identify recursive structure
Implementation Complexity	More complex — requires queue, visited set, state encoding	Simpler — short recursive function
Closed-form Solution	Confirmed by BFS, not derived analytically	Yes: $M(n) = (2/3) \cdot 2^n - (1/6) \cdot (-1)^n - 1/2$
Practical Advantage	Useful for general puzzles; verifies the optimal solution	Efficient both in time and memory for large n



7 CONCLUSION

Overall, both approaches confirm that the problem has exponential time complexity, but they differ in methodology: BFS is suitable for explicit state-space exploration, whereas the Divide and Conquer approach is better for mathematical modeling and theoretical analysis of performance. Although both approaches exhibit exponential growth, BFS incurs additional polynomial overhead due to explicit state exploration, whereas the divide-and-conquer approach directly constructs the optimal solution.

8 REFERENCES

- [1] A. Levitin, *Algorithmic Puzzles*, New York, NY, USA: Oxford University Press, 2011.
- [2] Saima Jawad, *Design and Analysis of Algorithms Lecture Notes*, Department of Computer Science, Bahria University, Islamabad.

9 ACKNOWLEDGEMENT OF THE IMPLEMENTATION

The algorithm design and logical formulation were developed independently by the student. The pseudocode was also written by the student. Python code implementation was generated with the assistance of a large language model and subsequently reviewed and validated by the student for correctness and consistency with the recurrence relations derived and theoretical analysis.